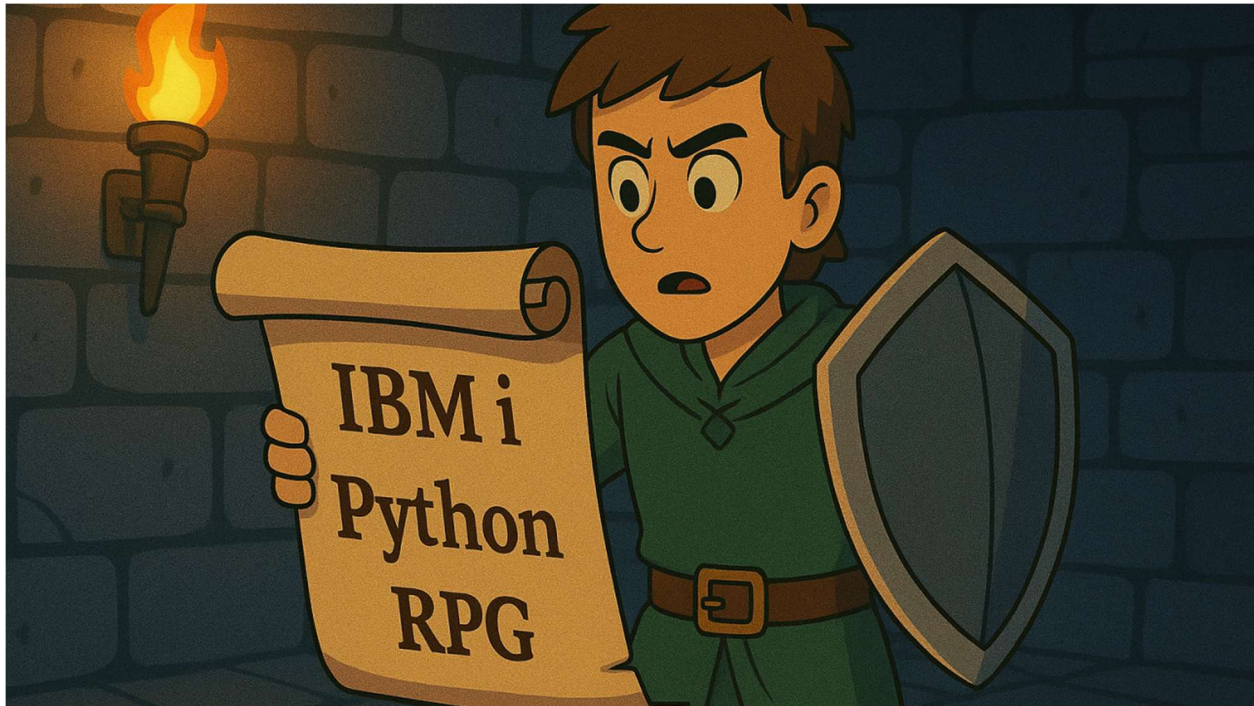


Python on the IBMi – Chapter 1



Disclaimer

Before we begin. I am not a seasoned Python developer. In fact, this is the first time past a "Hello World" program that I've tried to write a Python program. This will be a series of articles that as I adventure into Python and the IBMi that I thought I would share what I learn. These articles might not be the best place to find Python and IBMi best practices. I would enjoy seeing feedback, comments, and suggestions from others and on how they can be improved. When I went looking for information about Python and the IBMi I didn't find a lot so I thought I would contribute my findings.

I will also include my source code in a GitHub Repo so that you can easily download it.

(https://github.com/thomprl/ibmi_python_rpg_adventures)

I also thought I would have a little fun with the articles so I hope you like my quirky sense of humor.

Introduction

Once upon a time on a platform not so far away, I set out on a quest. With a keyboard as my sword, a command line as my shield, and a curious spirit guiding me, I stepped into the vast and sometimes mysterious world of Python, RPG, and the wizardry of the IBMi.

This isn't a tale of a seasoned wizard with decades of scrolls and spells under his belt. No, I'm more of a swordsman, eager, a bit unsure, but determined to learn. Along the way, I'll battle cryptic error messages, outwit mischievous bugs, and hopefully discover hidden treasures of knowledge buried.

If you've ever wondered what it's like to mix Python's flexibility with RPG's reliability on the IBMi, you're welcome to join me on this journey. We'll begin with the humble "Hello, World," and then venture into calling RPG programs from Python. Who knows what dangers we'll face or challenges that will present themselves along the way.

So grab your gear, fellow traveler. The path ahead is uncertain, but the adventure has just begun.

Prerequisites

To follow along, you'll need Python and iToolkit installed on your IBMi. I'll admit I haven't done the installation process myself before, as a system administrator usually handles that at the places I've worked. However, you can use the **Open Source Package Management** in Access Client Solutions (ACS) or use yum. A lot of this information can be found from our good friends over at Seiden Group (<http://seidengroup.com>)

- VS Code with the Code for IBMi extension (optional, but this is the way!)
- Python Installed (<https://ibmi-oss-docs.readthedocs.io/en/latest/python/README.html>)
- Bash set-up as your default shell (https://ibmi-oss-docs.readthedocs.io/en/latest/troubleshooting/SETTING_BASH.html)
- Set your path (https://ibmi-oss-docs.readthedocs.io/en/latest/troubleshooting/SETTING_PATH.html)
- iToolKit (<https://github.com/IBM/python-itoolkit>)

Objective

- Verify Python is working on the IBMi with a simple hello world program.
- Call an RPG program from Python, passing parameters and retrieve a parameter back.
- Call an RPG program with a VARCHAR as a parameter.

Let's Code

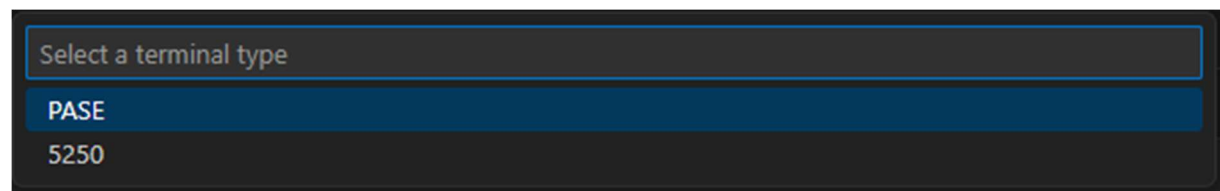
If you already know Python is working on your system, feel free to skip this part. If not, let's test it.

Create a folder on the IFS—for example: (**/home/rthompson/python/chapter1**)

Then, inside that folder, create a file called (**helloworld.py**) containing:

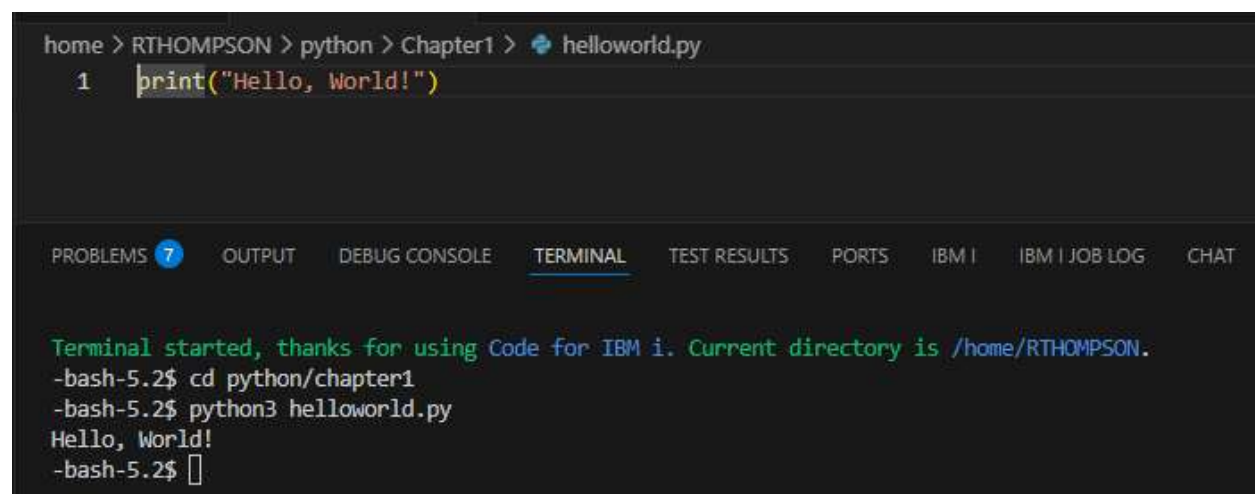
```
print("Hello, World!")
```

Open VS Code and press (Ctrl+Shift+J) to open a PASE terminal session. It will give you an option to open a PASE or 5250 session. Select PASE.



Since my current directory when I start my PASE session is already (**/home/rthompson**) I change to the directory where I placed my [helloworld.py](#) file. And run the program. The results should show Hello World! Like the screenshot below.

```
cd python/chapter1  
python3 helloworld.py
```



Calling an RPG Program

I came across this helpful article by Mike Larsen on ITJungle

(<https://www.itjungle.com/2020/03/30/guru-calling-rpg-programs-from-python-part-1/>). It demonstrates calling an RPG program from Python. However, it used a deprecated module:

itoolkit.lib.ilibcall. Instead, I used *itoolkit.transport.DirectTransport*, which is now recommended.

I'm going to show two examples of calling an RPG program with the included Python and RPG code. The first example shows how to include the library in the call statement and passing parameters. The second shows how to run a command to add the library to the library list but also shows how to pass a very large VARCHAR.

EXAMPLE1

Inside this folder (**/home/rthompson/python/chapter1**) I created a new file called [example1.py](#) with the following code.

```
from itoolkit import *
from itoolkit.transport import DirectTransport
itransport = DirectTransport()

itool = iToolKit()

itool.add(
    iPgm('rpgle_results', 'EXAMPLE1', {'lib':
'RTHOMPSON1'})
    .addParm(iData('InParm','20a','IBMi'))
    .addParm(iData('OutParm','200a',' '))
)

itool.call(itransport)

# results are returned as a dictionary formatted as Json

rpgle_results = itool.dict_out('rpgle_results')

if 'success' in rpgle_results:
    print(rpgle_results)
else:
    print('Errors occurred.')

# parse the Json response from the dictionary

print('\n')
print("OutParm: ", rpgle_results['OutParm'])
print("Status: ", rpgle_results['success'])
```

The RPG code looks like the following and is compiled to library RTHOMPSON1. The RPG program is called EXAMPLE1.RPGLE

```

**free
ctl-opt Main(Main) DftActGrp(*No);
dcl-ds psds PSDS qualified;
  pgmname *PROC;
  pgmsts zoned(5:0);
  pgmprvsts zoned(5:0);
  pgmsrcstmt char(8);
  pgmroutine *ROUTINE;
  pgmparms *PARMS;
  pgmmsgid char(7);
  pgmmi# char(4);
  pgmwork char(30);
  pgmlib char(10);
  pgmerrdta char(80);
  pgmrpgmsg char(4);
  Filler_01 char(69);
  pgmjob char(10);
  pgmuser char(10);
  pgmjobnum zoned(5:0);
  pgmjobdate zoned(6:0);
  pgmrundate zoned(6:0);
  pgmruntime zoned(6:0);
end-ds psds;

dcl-proc Main;
  dcl-pi *n;
    InParm varchar(500000:4) const;
    OutParm varchar(500000:4);
  end-pi;

  OutParm = 'Welcome ' + %trim(InParm)
    + ', to Python/RPGLE programming! - '
    + 'Job Name: '
    + %editc(psd.pgmjobnum:'X')
    + '/' + %trim(psd.pgmuser)
    + '/' + %trim(psd.pgmjob);
end-proc Main;

```

From VS Code Open a PASE terminal by pressing (**Ctrl+Shift+J**) and selecting PASE and then type the following:

```

cd python/chapter1
python3 example1.py

```

EXAMPLE2

The second example will look very similar to the first but the incoming and outgoing parameters will be a large VARCHAR(500000:4) and it runs a command to add the library first before calling instead of coding the library in the call command to just show how to do it.

Here is the Python code for example2.py

```
from itoolkit import *
from itoolkit.transport import DirectTransport
itransport = DirectTransport()

itool = iToolkit()

itool.add(iCmd('addlibl', 'addlibl rthompson1'))

itool.add(
    iPgm('rpgle_results', 'EXAMPLE2')
    .addParm(iData('InParm', '500000a', 'IBMi', {'varying':
4}))
    .addParm(iData('OutParm', '500000a', ' ', {'varying':
4}))
)

itool.call(itransport)

# results are returned as a dictionary formatted as Json

rpgle_results = itool.dict_out('rpgle_results')

if 'success' in rpgle_results:
    print(rpgle_results)
else:
    print('Errors occurred.')

# parse the Json response from the dictionary
print('\n')
print("OutParm: ", rpgle_results['OutParm'])
print("Status: ", rpgle_results['success'])
```

And in the RPG code for EXAMPLE2.RPGLE I changed the Procedure Interface to
varchar(500000:4)

```
**free
ctl-opt Main(Main) DftActGrp(*No);
dcl-ds psds PSDS qualified;
```

```

pgmname *PROC;
pgmsts zoned(5:0);
pgmprvsts zoned(5:0);
pgmsrcstmt char(8);
pgmroutine *ROUTINE;
pgmparms *PARMS;
pgmmsgid char(7);
pgmmi# char(4);
pgmwork char(30);
pgmlib char(10);
pgmerrdta char(80);
pgmrpgmsg char(4);
Filler_01 char(69);
pgmjob char(10);
pgmuser char(10);
pgmjobnum zoned(5:0);
pgmjobdate zoned(6:0);
pgmrundate zoned(6:0);
pgmruntime zoned(6:0);
end-ds psds;

dcl-proc Main;
  dcl-pi *n;
    InParm varchar(500000:4) const;
    OutParm varchar(500000:4);
  end-pi;

  OutParm = 'Welcome ' + %trim(InParm)
    + ', to Python/RPGLE programming! - '
    + 'Job Name: '
    + %editc(psds.pgmjobnum:'X')
    + '/' + %trim(psds.pgmuser)
    + '/' + %trim(psds.pgmjob);
end-proc Main;

```

Now run the Python program example2.py by typing the following in the PASE terminal

```
python3 example2.py
```

You should see basically the same results as example1.py but it's calling using a VARCHAR and adding the library to the library list.

```

-bash-5.2$ python3 example2.py
{'InParm': 'IBMI', 'OutParm': 'Welcome IBMI, to Python/RPGLE programming! - Job Name: 79812/QUSER/QP0ZSPMT', 'success': '+++ success  EXAMPLE2'}

OutParm: Welcome IBMI, to Python/RPGLE programming! - Job Name: 79812/QUSER/QP0ZSPMT
Status: +++ success  EXAMPLE2
-bash-5.2$

```

What's Next

With his first lines of Python carved and “Hello, World” echoing through the forest, our hero takes his first real steps beyond the safety of the trail. Ahead lies a dark cave, with the word *pyodbc* etched into the stone above its entrance a place whispered about in legends as the gateway to DB2 and the data that lies within.

Will he dare to enter?

Join us in Chapter 2 as the journey leads deeper into the mysteries of SQL and the IBMi. So, gather your armor, spell book, and trusty keyboard our adventure is just beginning.

Stay tuned!